

Autonomous Systems Project Report:

Predictive Human Flow Navigation with Automatic Maintenance

Team Members:

Mahmoud Bassam Elnaqa – 19001991

Nitish Tukaram – 16001673

Conception and Requirements:

As stated more than 4 months ago, a problem persists in the field of Autonomous Driving where cars will only avoid pedestrians upon detecting them in close proximity. This project aims to improve on that mechanism by predicting where these pedestrians are projected to go and then creating alternative paths to avoid them in advance.

Autonomous maintenance mechanisms, such as automatic charging upon the battery's near death and automatic repair upon near degradation, are to be implemented in this project as well.

Architecture and Design:

The project was designed in the following phases:

Phase 1 → Set up and organize ROS2 environment, installing Gazebo, TurtleBot, Nav2, RViz, and their dependencies

Phase 2 → SLAM Mapping and Localization throughout the TurtleBot arena

Phase 3 → Design and implementation of Nav2 goal navigation and

recovery behaviour functions

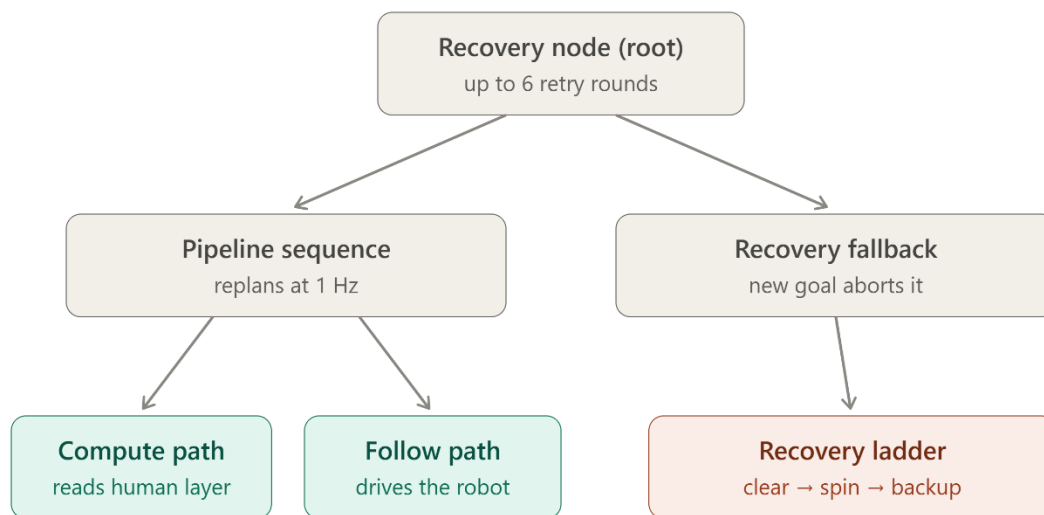
Phase 4 → Human Detection and Predictive Occupancy Grid

Phase 5 → Failure-aware system and Self-Monitoring

Phase 6 → Task-Driven Behaviour using object detection and predictive costmap.

Please note that only Phases 1 to 5 have been implemented due to time constraints caused by poor project time management.

In Phase 1, the workspace was prepared for the project through the installation of the ROS2 libraries and dependencies mentioned. In Phase 2, SLAM was implemented by running the SLAM toolbox in a manual, asynchronous manner with the robot through the entire TurtleBot arena, in addition to the configuration of the AMCL for localization. In Phase 3, a Nav2 goal planning, navigation, and recovery system was designed based on the following behaviour tree:

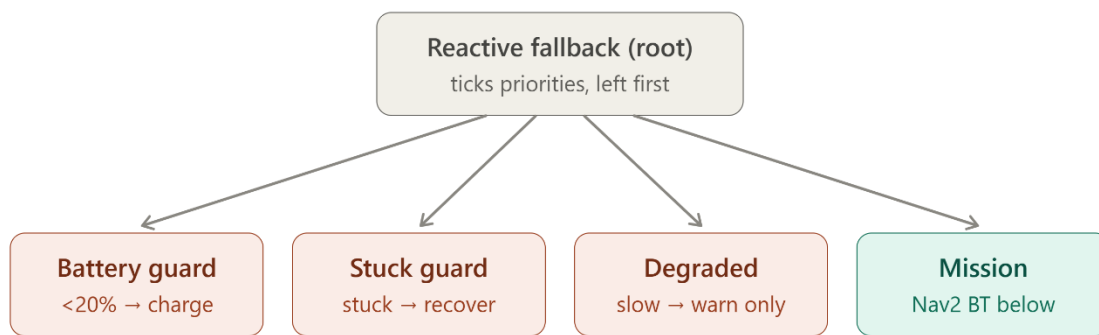


Gray = control flow · Teal = navigation actions · Coral = recoveries (then wait 5 s)

In Phase 4, a human was put in the world at the farthest column, and a human prediction system in the form of the “human_predictor” package, was created. This system/package can be reviewed and run through the

“human_predictor.py” file in

~/autonomous_mobile_robot_ws/src/navigation_manager/navigation_manager. Afterwards, in Phase 5, a failure-aware system which monitors battery life and degradation levels was designed and implemented, and the instructions to run that final system has been written in the README file attached. The behaviour tree for that system can also be seen below:



Coral = supervisor guards (failure_monitor FSM) · Teal = Nav2 navigation

Detailed Implementation, Testing. and Error

Debugging:

At the end of every phase implemented in the project, the deliverables of each phase were tested as follows:

Phase 1 → No real testing was needed apart from making sure that the installed ROS2 libraries and programs worked as intended.

Phase 2 → RViz was checked to make sure that the map generated by the SLAM toolbox showed up as a frame before AMCL localization could be implemented.

Phase 3 → The implemented system in that phase was first tested using manual Nav2 2D pose estimate and Nav2Goal selectors. The

navigate_to_pose action server refused to load at first due to a version mismatch problem, which was resolved by updating and upgrading the ROS2 Nav2 Route library and its dependencies. After that, the path to the goal was printed out successfully, however, the robot wouldn't move. This was resolved by adding "enable_stamped_cmd_vel: true" on every server/function in the nav2_params.yaml that was related to the robot's velocity. Robot would also oscillate and jerk while moving. This was resolved by killing all running processes and re-running them again on a clean system. Finally, the robot was able to navigate towards the goal with recovery mechanism working as intended. A goal_navigator.py script package was then created and tested to make sure that the robot can estimate initial pose and navigate automatically towards a preset goal without any flaws, and the algorithm worked as intended.

Phase 4 → After addition of the walking human actor to the world, the actor was edited several times because he was sunk in order to make sure that only his feet are touching the ground, then the human predictor script package was written. After running the simulation stack and the human predictor, the robot's AMCL servers would time out so the robot could not set a goal nor navigate towards the preset goal from the goal_navigator.py in Phase 3. This was resolved by initializing the 2D robot pose as (0.0,0.0,0.0) on the nav2_params.yaml so that the AMCL does not time out while estimating that pose. Finally, the human prediction system worked and the robot could navigate towards the goal, optimizing its path to make sure it avoids the human entirely during navigation, which was the core function of our project.

Phase 5 → A battery_simulator.py script package for simulating the robot's battery was added in addition to a failure_monitor.py script package for monitoring robot status, the most notable states being NOMINAL, RECOVERY, LOW BATTERY, and FAILURE. The system

was ran first without the human predictor, however, the robot could neither set the goal nor navigate towards it, so the stack was re-run again as described in the final README file attached, and the robot was able to navigate successfully towards the goal while implementing recovery mechanisms in near-collision cases, with the failure monitor showing the states correctly. When the robot's simulated battery life fell to 20%, its failure monitor state changed to LOW BATTERY and the robot tried to navigate towards the charging station, but since its coordinates were set as (0.0,0.0), the algorithm interpreted it as in the middle or near a site of potential collision. Therefore, the coordinates were set to (2.0,2.0) in the README file to prevent such disaster when tested by any potential reviewers.

Conclusions and Future Work:

Conclusions:

- The pedestrian-predicting autonomous car system was implemented in 5 phases, with a 6th optimization phase that was designed in the project not being implemented due to poor time management.
- Despite the challenges mentioned above for each phase, the deliverable systems (or mini-systems) of each phase worked as intended, with the exception of Phase 5 since the battery_simulator and failure_monitor script packages were run with incompatible charging station coordinates, and we could not fix that problem due to time constraints caused, once again, by our poor project time management.
- The project was implemented for a scenario of navigating from an initial location to a preset goal, however, that is not enough since a scenario involving multiple goals needs to be addressed as well.

Future Work:

- Implementation of Phase 6 → Task-Driven, object detection based behaviour optimization is crucial to the development of advanced autonomous systems, and implementing that phase would greatly improve the project and help the robot adapt to any environment, in addition to Phase 6 being a pre-requisite for hardware integration.
- Pre-determine coordinates of key objects and points in the TurtleBot arena → Pre-determining coordinates of key objects and points in the TurtleBot arena, and setting a fixed, preset charging/repair station coordinates would prevent and fix the error caused by incompatible charging station coordinates.
- Implement a navigation script involving multiple goals → Implementing and testing a navigation scenario involving multiple goals helps analyze and improve the accuracy of the predictive autonomous robot when it comes to predictively navigating in order to avoid human obstacles (pedestrians), and thus this helps prepare said robot for real-world tasks and scenarios.